

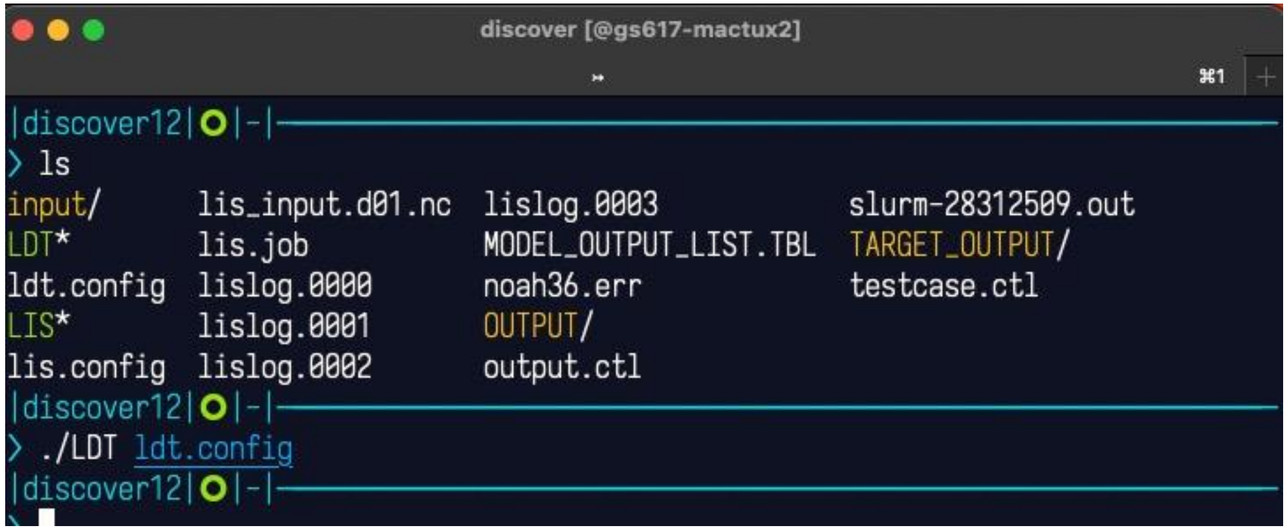
Configuring LISF and Reading Error Messages

James Geiger
NASA/GSFC Code 587/617

Configuring LISF

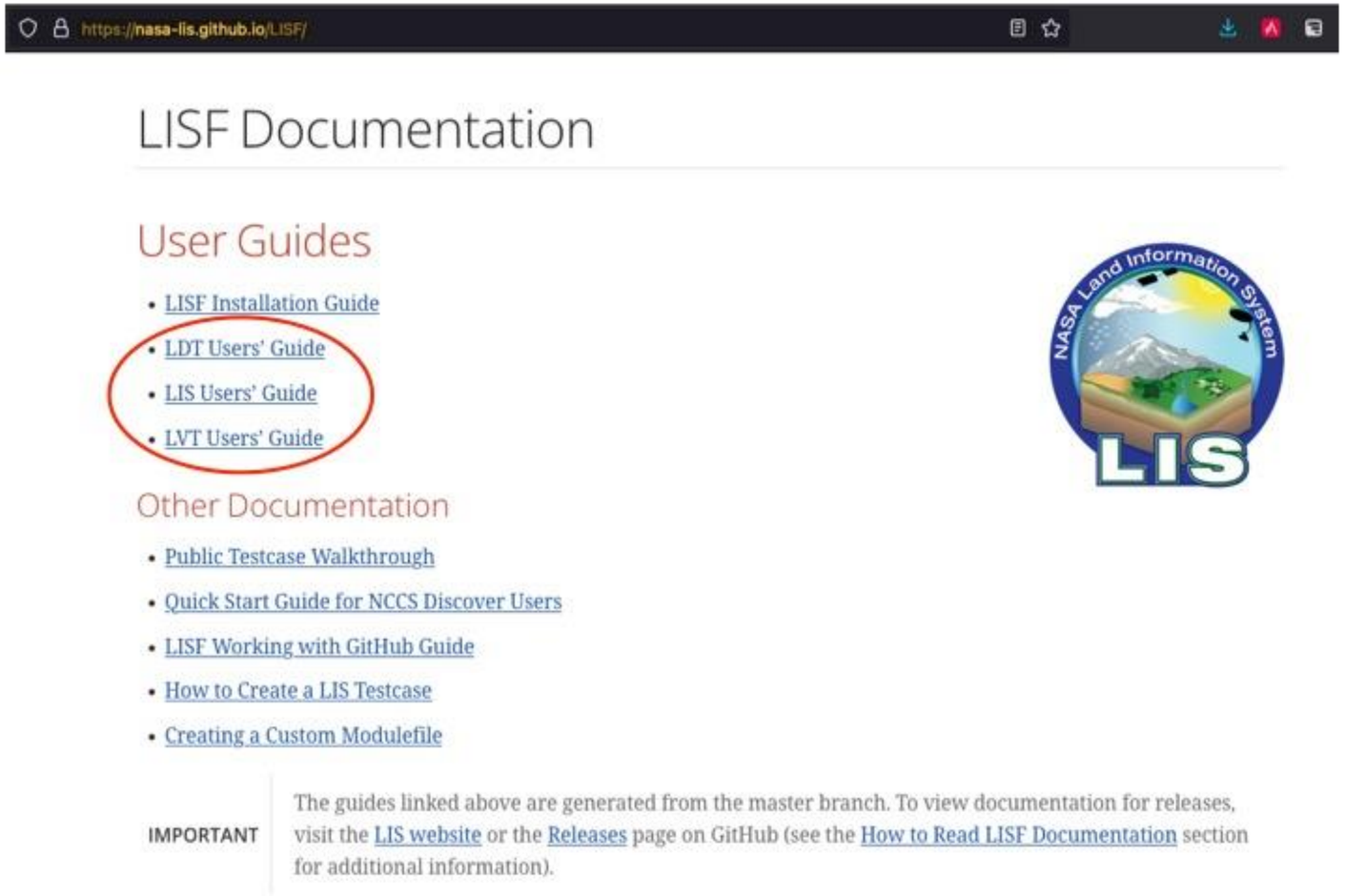
Configuring LISF

LDT, LIS, and LVT are command line programs. They are run at the command line/shell prompt or via a shell script.



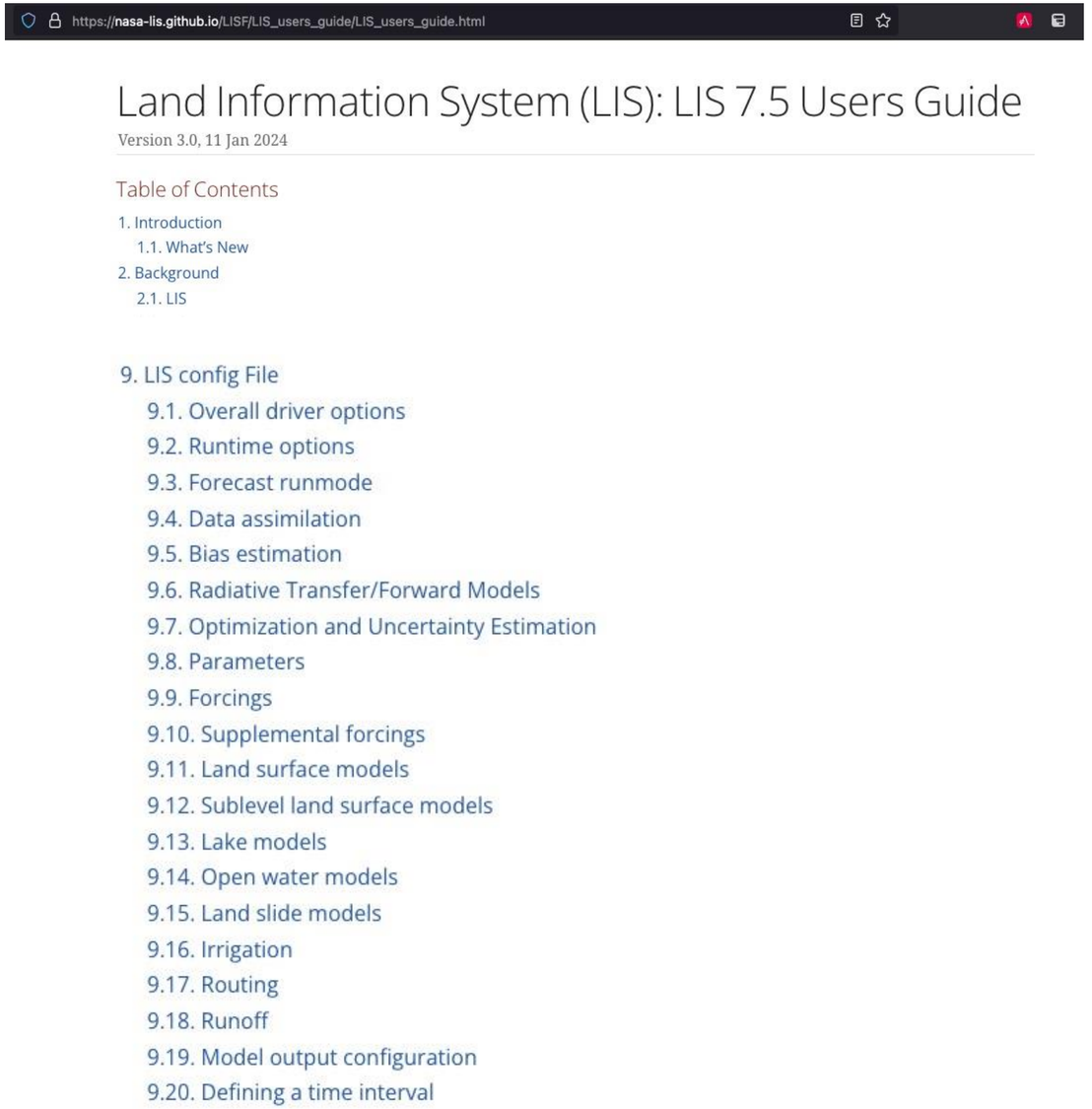
They are configured via text files – not via a GUI.

Configuring LISF



Configuring LISF

We will look at the *lis.config* file used to configure LIS. The *LIS Users Guide* has a section that describes all the options available, similarly for LDT and LVT.

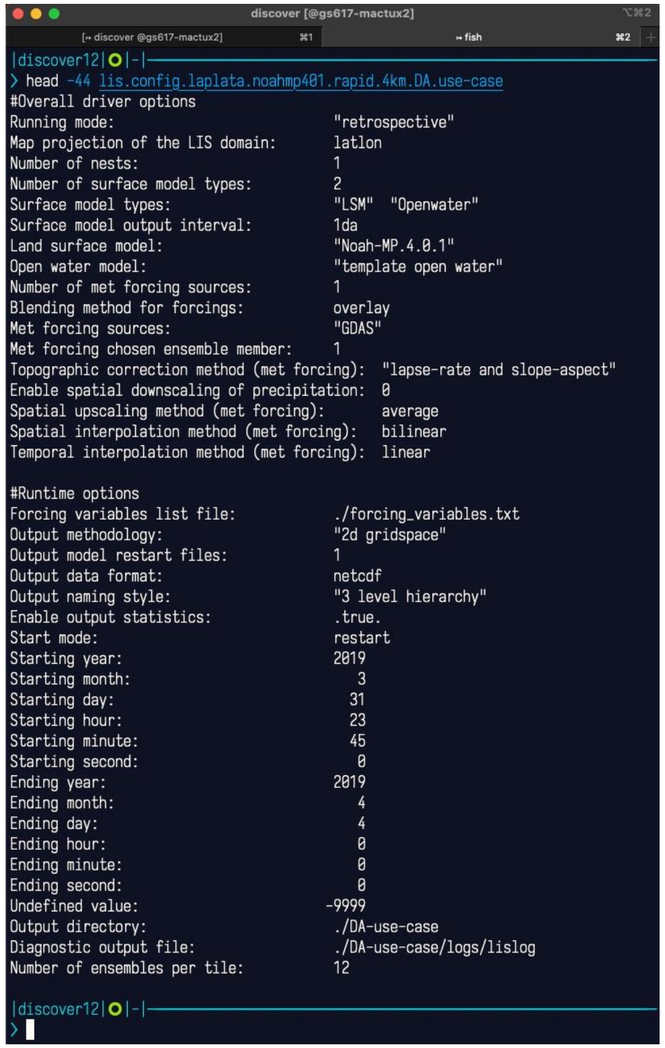


Configuring LISF

Here are the top few lines of a configuration file used to run LIS for this workshop.

We will look at a few of these options to learn how to configure LIS.

LDT and LVT are configured similarly.

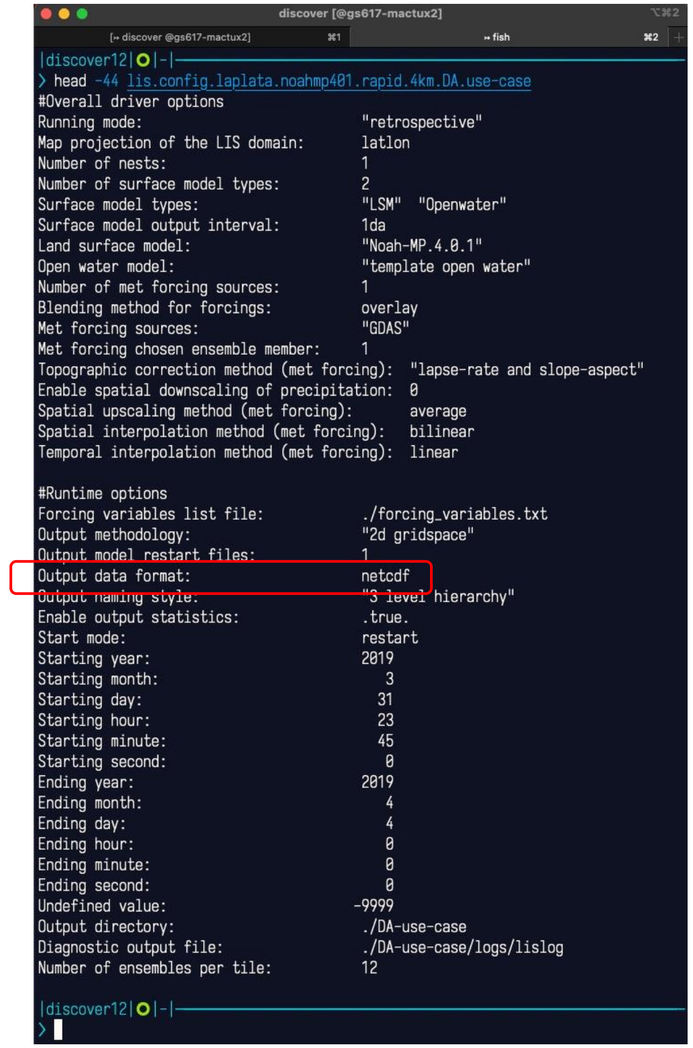


Configuring LISF

First, let's look at `Output data format:`.

This is an example of a setting with specific acceptable values.

Searching for `Output data format:` in the *LIS Users Guide* gives…



GODDARD EARTH SCIENCES

Configuring LISF

`Output data format`: specifies the format of the model output data. Acceptable values are:

Value	Description
“binary”	Write output in binary format
“distributed binary”	Write output in distributed binary format where each processor writes outputs for the respective local domain
“grib1”	Write output in GRIB-1 format
“grib2”	Write output in GRIB-2 format
“netcdf”	Write output in netCDF format

Example lis.config entry

```
Output data format: netcdf
```

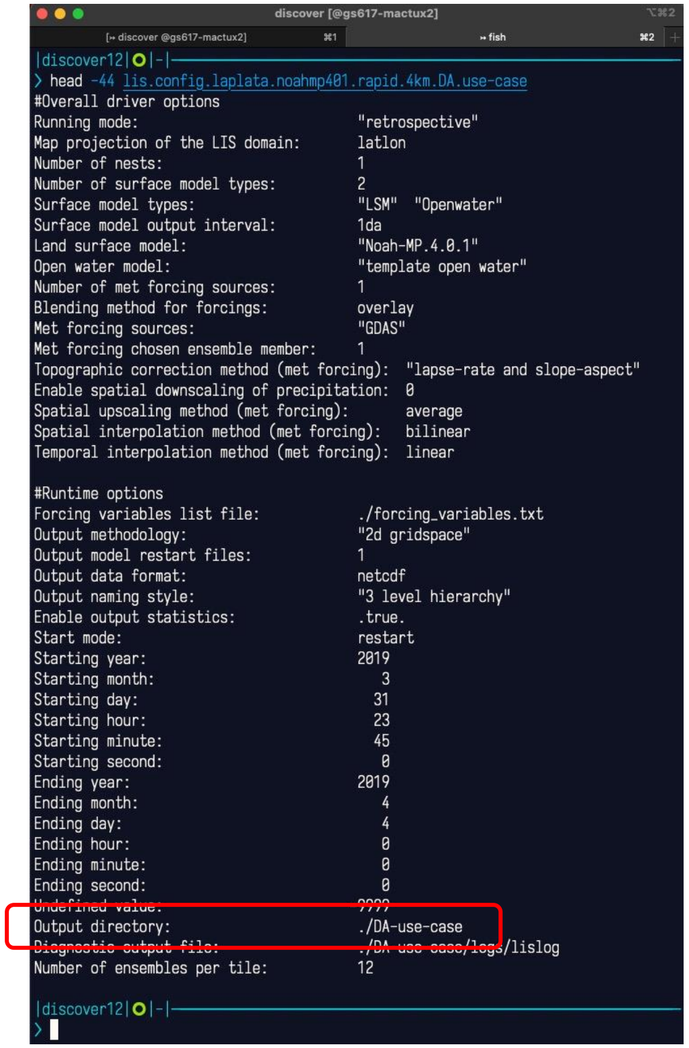
GODDARD EARTH SCIENCES

Configuring LISF

First, let's look at `Output directory:`.

This is an example of a setting with a user-specified value.

Searching for `Output directory:` in the *LIS Users Guide* gives…



GODDARD EARTH SCIENCES

Configuring LISF

`Output directory:` specifies the name of the top-level output directory. Acceptable values are any 40 character string. The default value is set to OUTPUT. For simplicity, throughout the rest of this document, this top-level output directory shall be referred to by its default name, *$WORKING/LIS/OUTPUT*.

Example lis.config entry

```
Output directory: OUTPUT
```

`Diagnostic output file:` specifies the name of run time diagnostic file. Acceptable values are any 40 character string.

Example lis.config entry

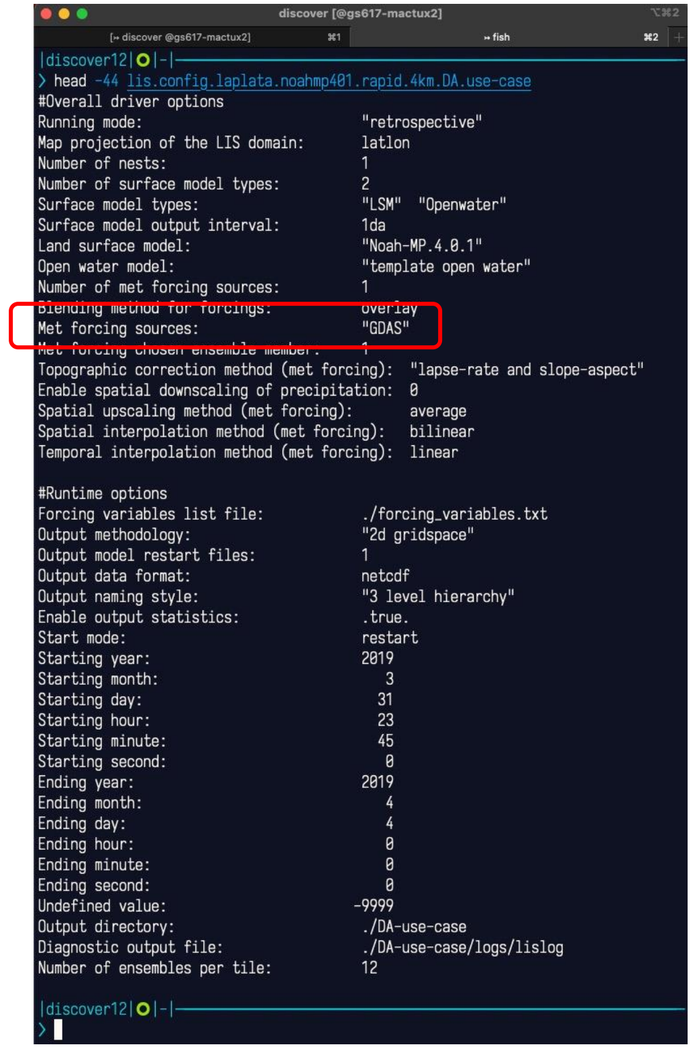
```
Diagnostic output file: lislog
```

Configuring LISF

Next, let's look at `Met forcing sources:`.

This is an example of a setting requires additional settings.

Searching for `Met forcing sources:` in the *LIS Users Guide* gives…



Configuring LISF

`Met forcing sources:` specifies the met forcing data sources for the run. The values should be specified in a column format. Acceptable values for the sources are:

Value	Description
“none”	None
“GDAS”	GDAS
“GEOS5 forecast”	GEOS5 Forecast
“GEFS forecast”	GEFS Forecast
“COAMPSout”	COAMPS output forcing
“GALWEM forecast”	GALWEM 17km or 0.25deg deterministic forecast reader
“GALWEM-GE forecast”	GALWEM-GE forecast reader

Example lis.config entry

```
Met forcing sources: GDAS
```

Configuring LISF

Continuing with `Met forcing sources:`, after selecting “GDAS”, you must also tell LIS where to find the files.

The *LIS Users Guide* has a section devoted to the specific settings required by each of the `Met forcing sources:` choices.



9.9. Forcings

9.9.1. GDAS

`GDAS forcing directory:` specifies the location of the GDAS forcing files.

Example lis.config entry

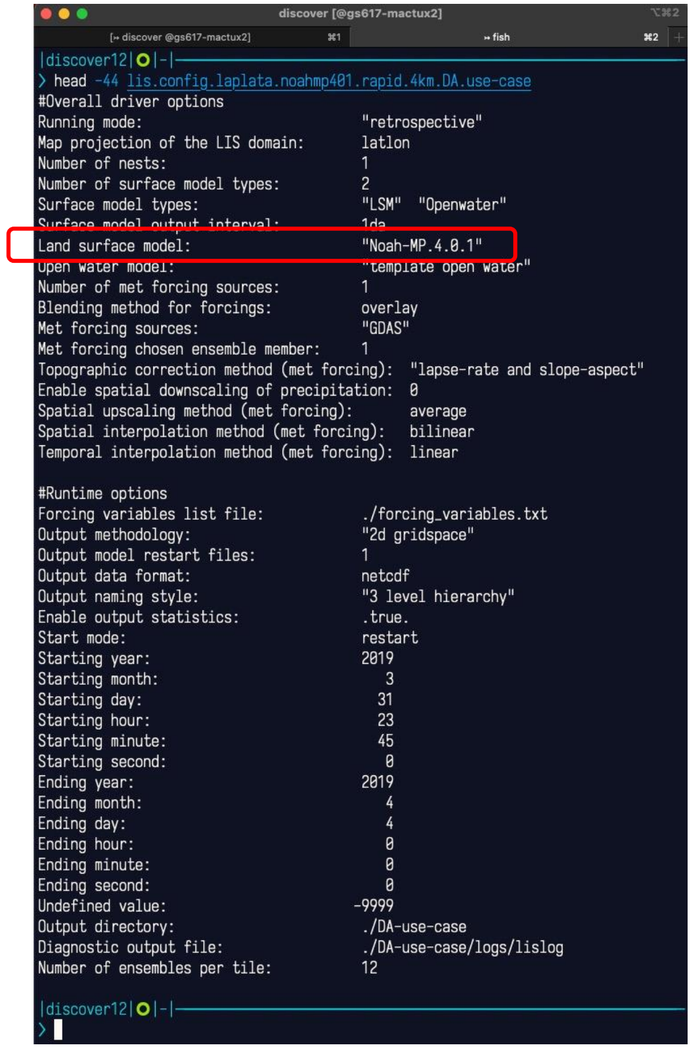
```
GDAS forcing directory:            ./input/FORCING/GDAS/
```

Configuring LISF

Finally, let's look at `Land surface model:`.

This is another example of a setting requires additional settings.

Searching for `Land surface model:` in the *LIS Users Guide* gives…



Configuring LISF

`Land surface model:` specifies the land surface model to run. Acceptable values are:

Value	Description
none	template lsm
Noah.2.7.1	Noah version 2.7.1
Noah.3.2	Noah version 3.2
Noah.3.3	Noah version 3.3
“CLSM F2.5”	Catchment Fortuna-2_5
RDHM.3.5.6	RDHM 3.5.6 (SAC-HTET and SNOW-17)
AWRAL.6.0.0	AWRA-L version 6.0.0

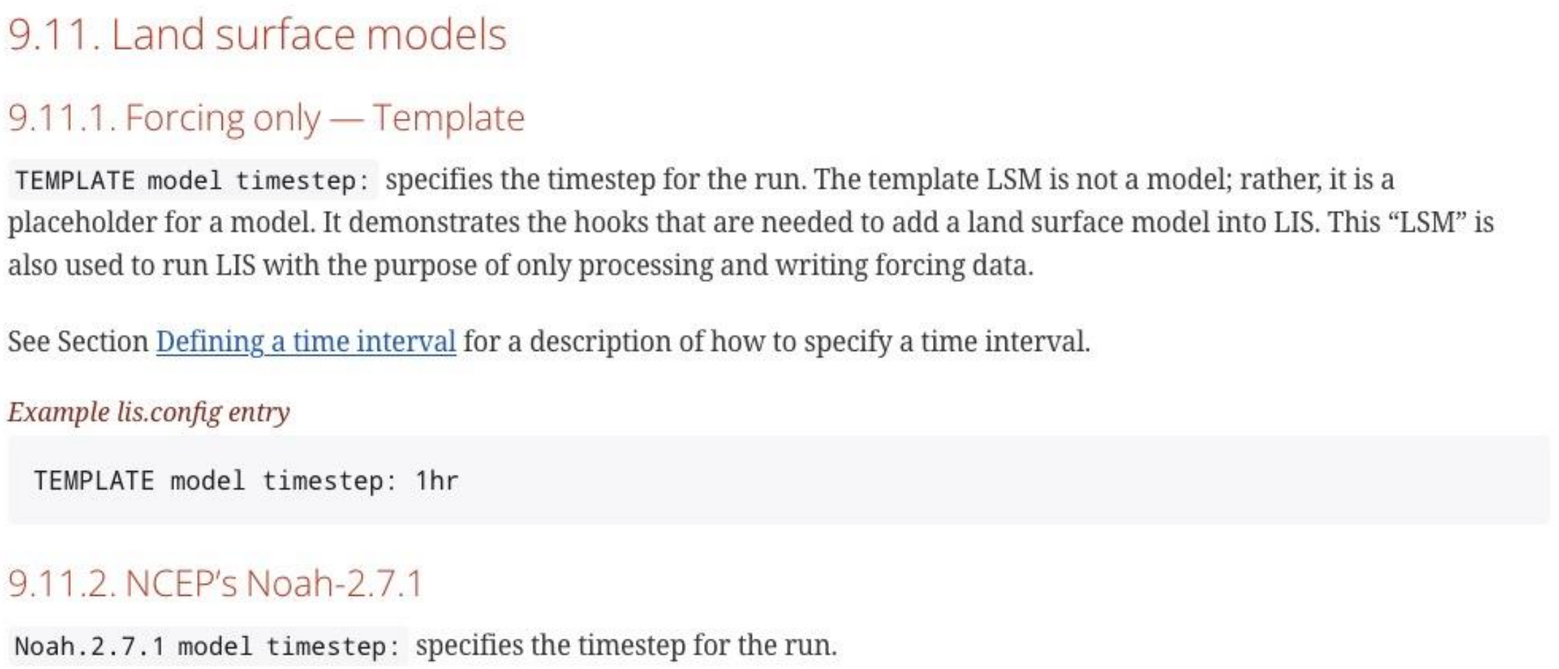
Example lis.config entry

```
Land surface model: Noah.2.7.1
```

Configuring LISF

Continuing with `Land surface model:`, after selecting “Noah-MP.4.0.1”, you must also tell LIS how to configure that land surface model.

The *LIS Users Guide* has a section devoted to the specific settings required by each of the `Land surface model:` choices.



9.11. Land surface models

9.11.1. Forcing only — Template

`TEMPLATE model timestep:` specifies the timestep for the run. The template LSM is not a model; rather, it is a placeholder for a model. It demonstrates the hooks that are needed to add a land surface model into LIS. This “LSM” is also used to run LIS with the purpose of only processing and writing forcing data.

See Section Defining a time interval for a description of how to specify a time interval.

Example lis.config entry

```
TEMPLATE model timestep: 1hr
```

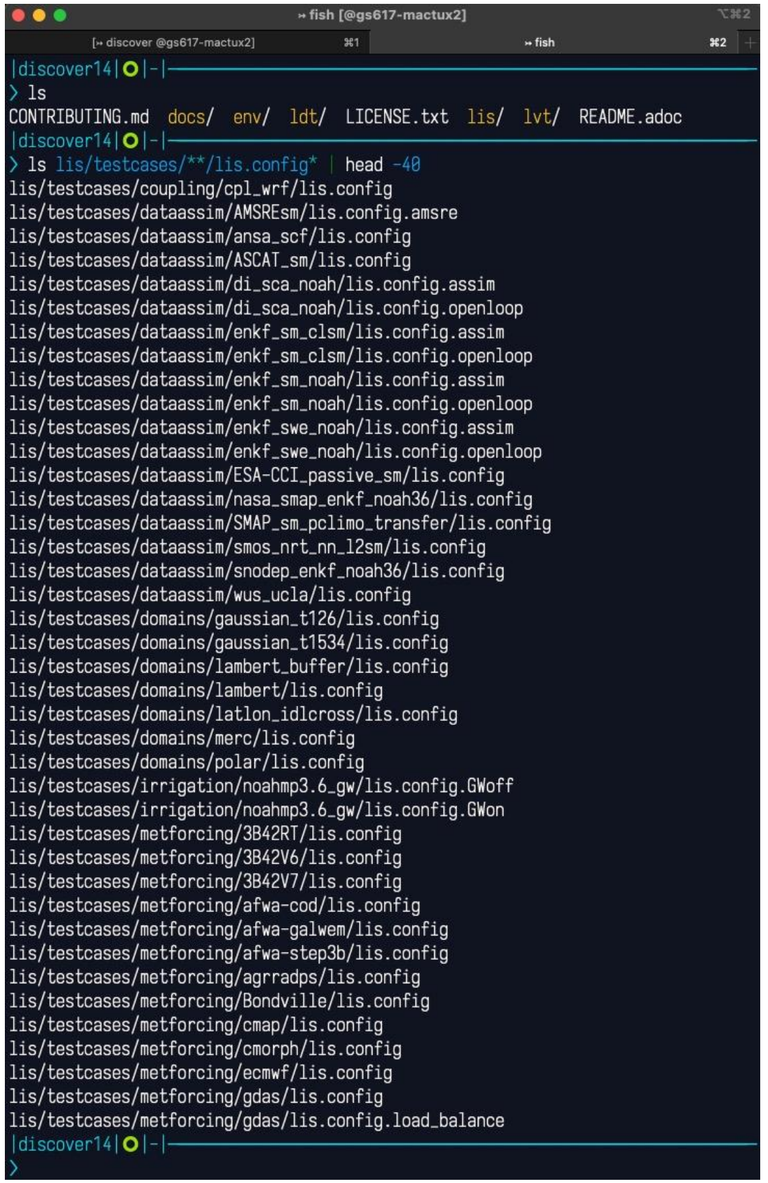
9.11.2. NCEP's Noah-2.7.1

`Noah.2.7.1 model timestep:` specifies the timestep for the run.

Configuring LISF

There are a lot of settings to make to run LIS. But you do not have to write a *lis.config* configuration file from scratch. You may take an existing file, like the one from this workshop, and tweak it to fit your needs.

There are example configuration files in the LIS source code. Look at the *lis.config* files in *lis/testcases/.*



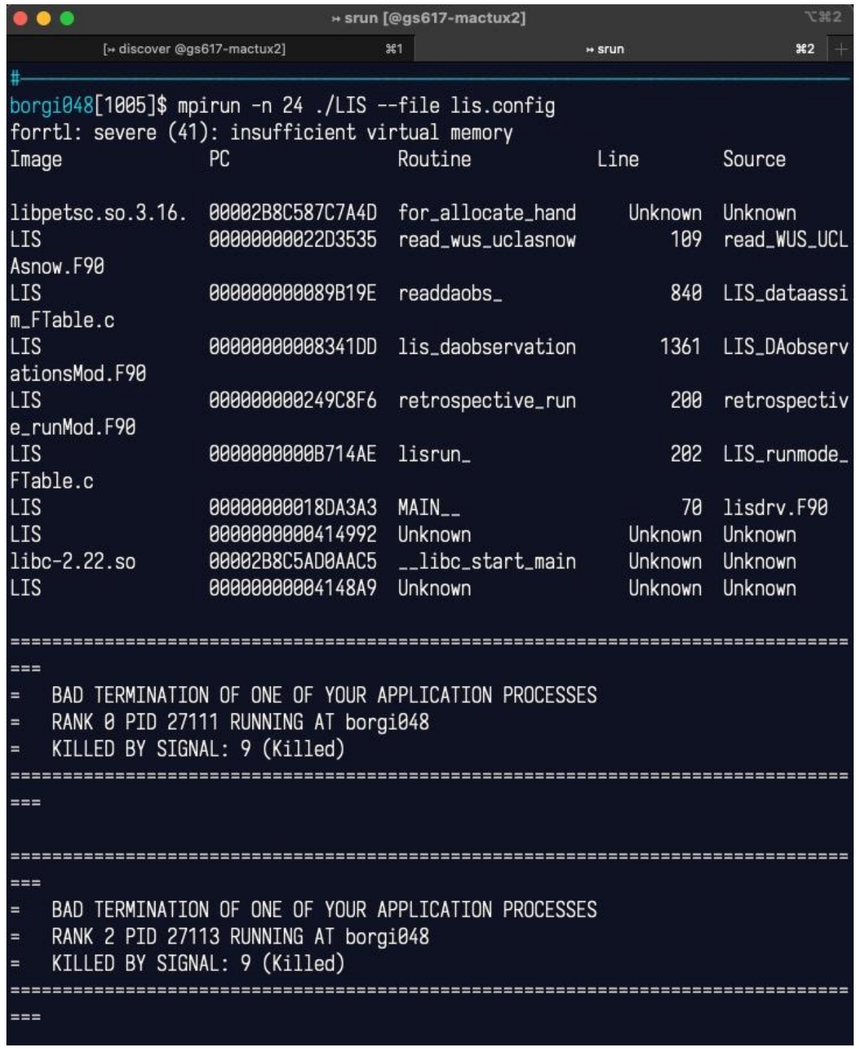
Error messages

Error messages

Sometimes, something goes wrong when running LIS. Usually, the operating system, run-time checks compiled into the executable, and/or LIS' own logging will catch problems and report them. These reports may be printed to your screen, captured in an error file, or written directly into LIS' log files. So, you may have to look in several places to fully understand the cause of a problem.

Error messages

This example shows several messages generated both by the OS and by the executable for a job that aborted.

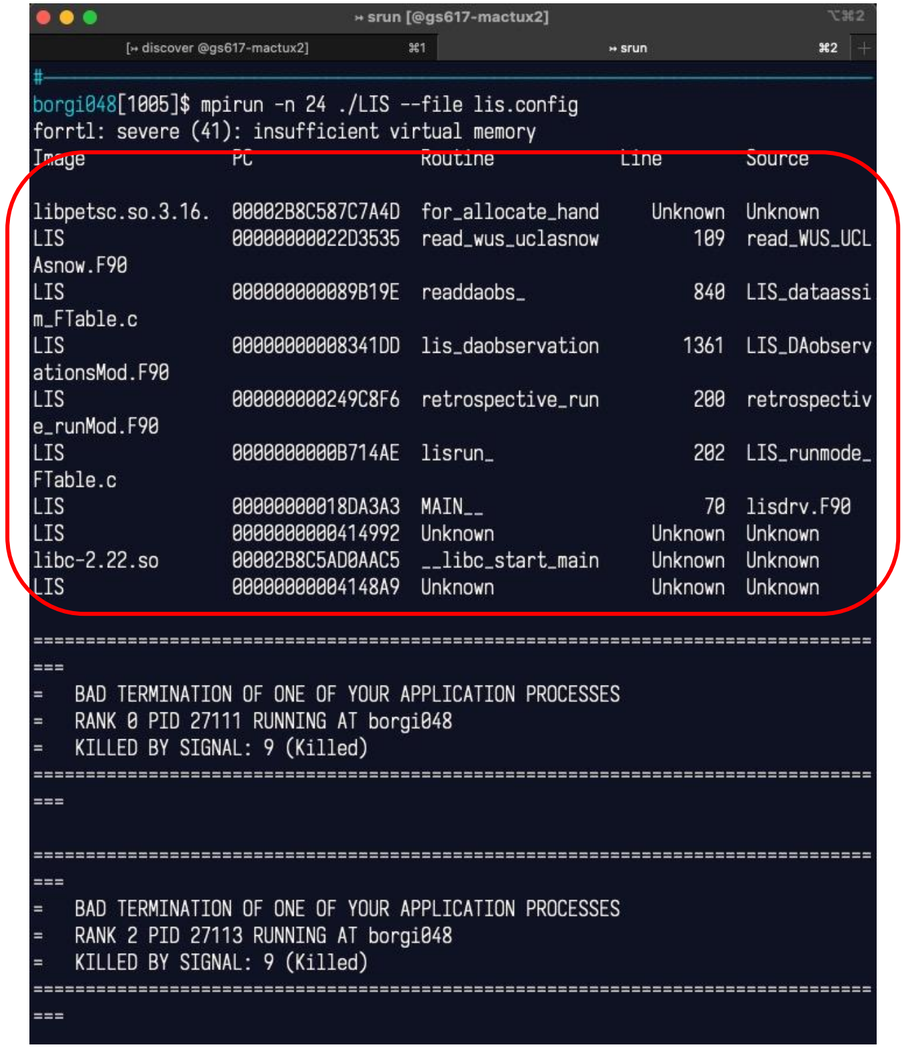


Error messages

This example shows several messages generated both by the OS and by the executable for a job that aborted.

This shows a stack trace (or a traceback). It shows you where in LIS the problem occurred.

Stack traces are most usefully when LIS is compiled with debugging enabled because it will provide more information. (See the *LIS Users Guide* for compilation options.)



Error messages

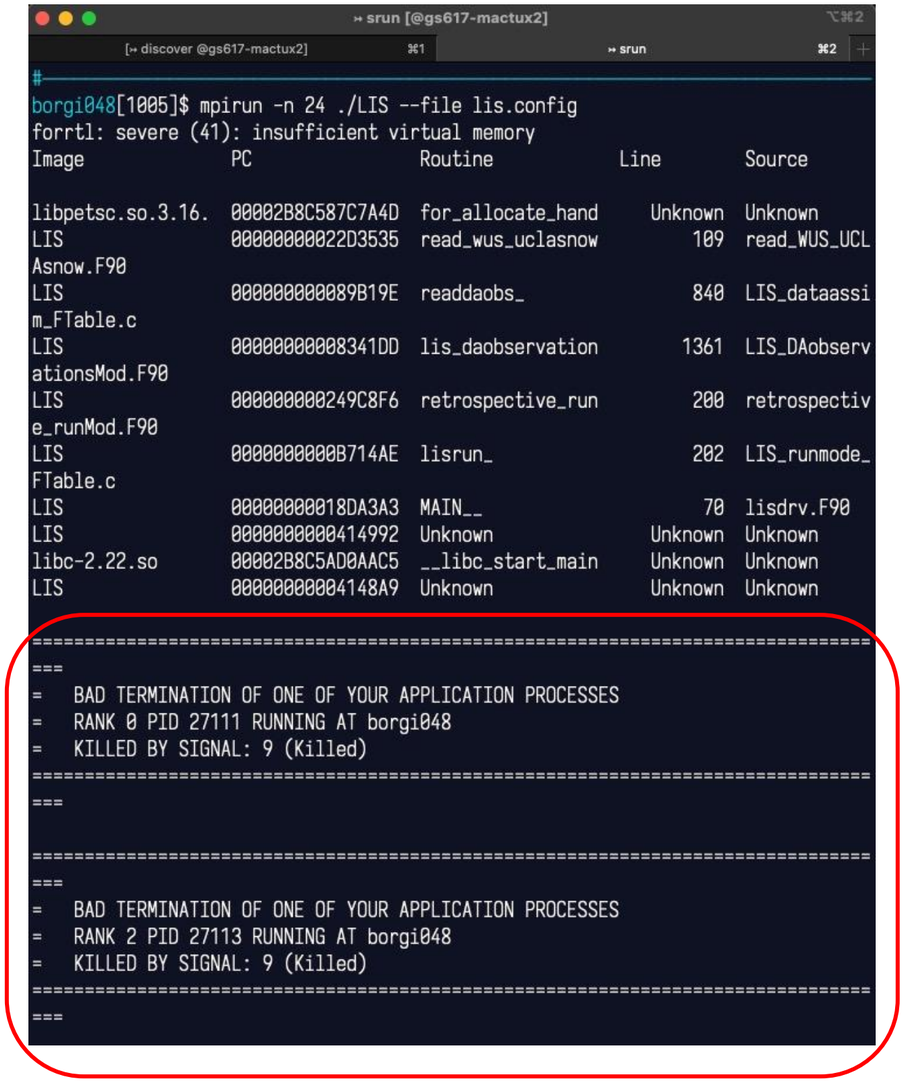
This example shows several messages generated both by the OS and by the executable for a job that aborted.

This shows a stack trace (or a traceback). It shows you where in LIS the problem occurred.

Stack traces are most usefully when LIS is compiled with debugging enabled because it will provide more information. (See the *LIS Users Guide* for compilation options.)

This shows some messages from the OS.

In this case, those messages are not very useful.



Error messages

This example shows several messages generated both by the OS and by the executable for a job that aborted.

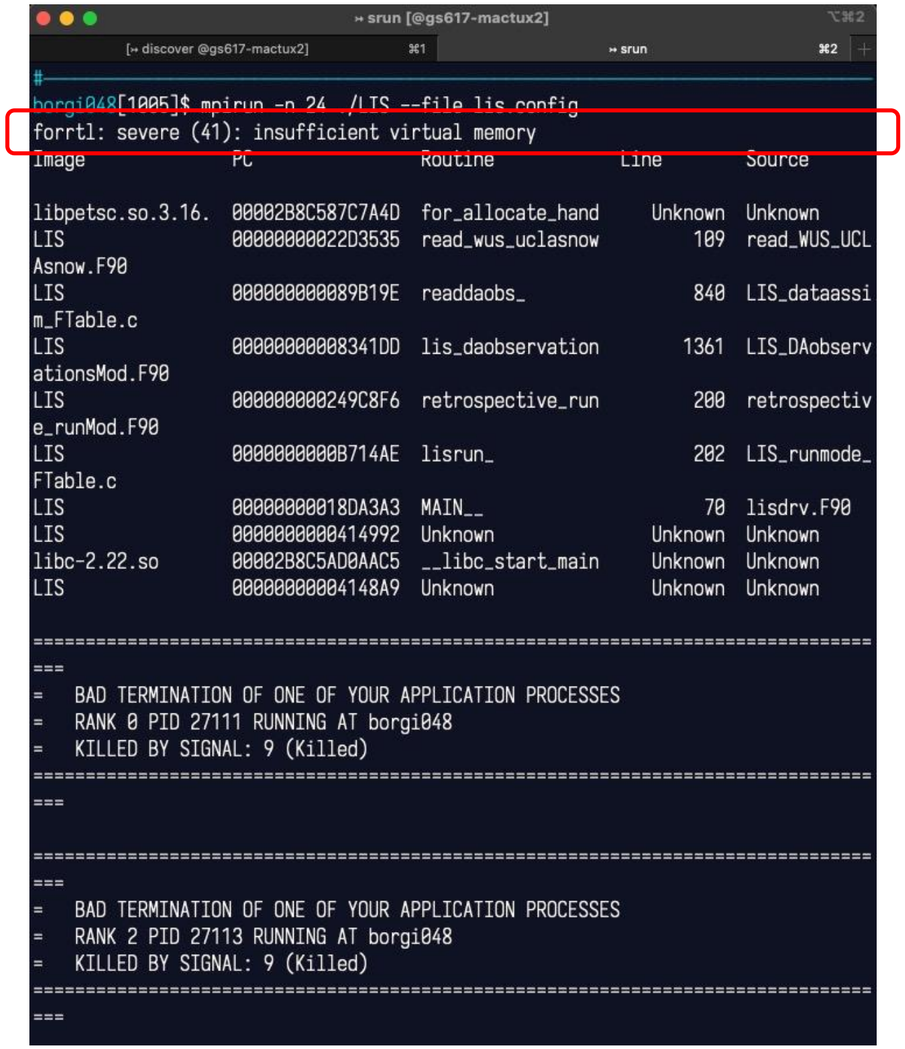
This shows a stack trace (or a traceback). It shows you where in LIS the problem occurred.

Stack traces are most usefully when LIS is compiled with debugging enabled because it will provide more information. (See the *LIS Users Guide* for compilation options.)

This shows some messages from the OS.

In this case, those messages are not very useful.

This message indicates the actual problem. My job ran out of memory. I must adjust my job to use a different number of nodes and tasks.



Error messages

For this example, I submitted a job into a queuing system (SLURM, PBS, etc.).
I submitted the job by running:

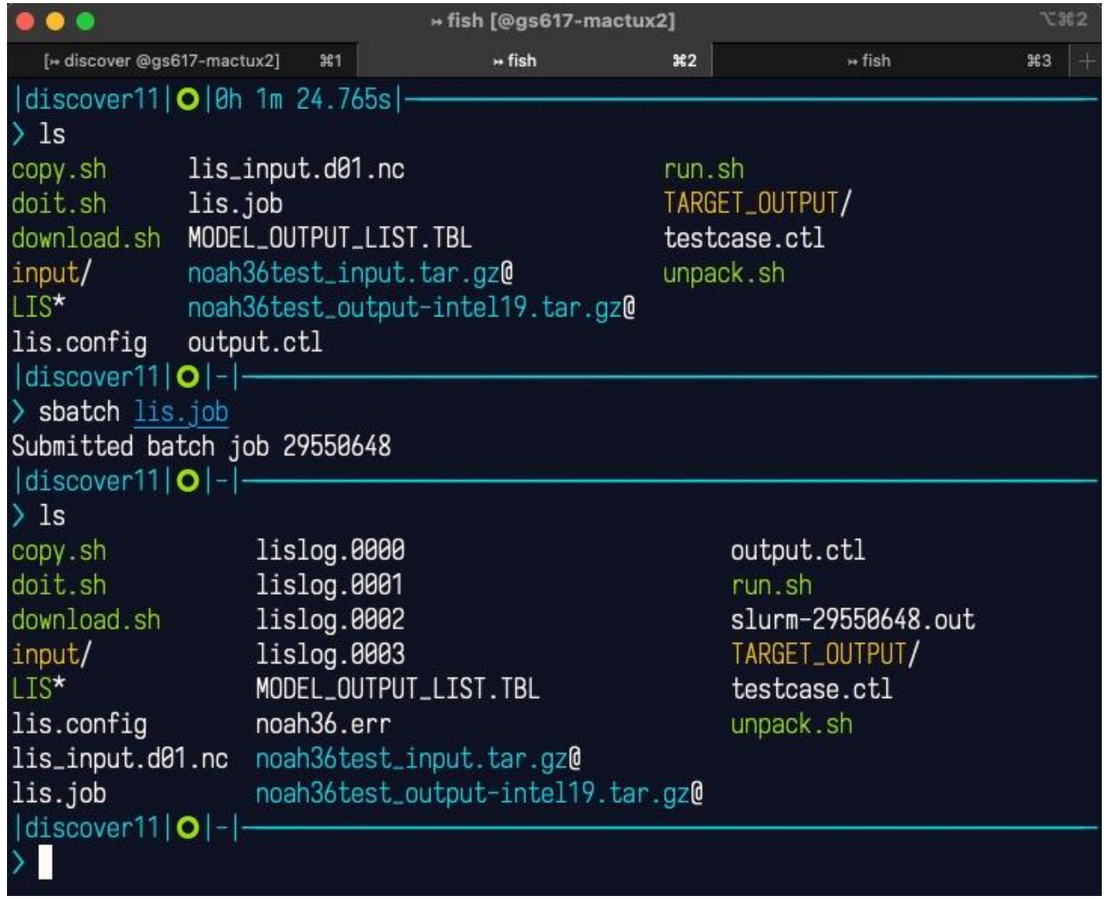
```
% sbatch lis.job
```

Where *lis.job* is this script:

```
#!/bin/bash
#SBATCH --job-name=noah36
#SBATCH --ntasks=4
#SBATCH --time=1:00:00
#SBATCH --account=s1189
#ADDITIONAL ARGUMENTS
#SBATCH --constraint="[sky|cas]"

echo $DEV_ENV
cd $SLURM_SUBMIT_DIR

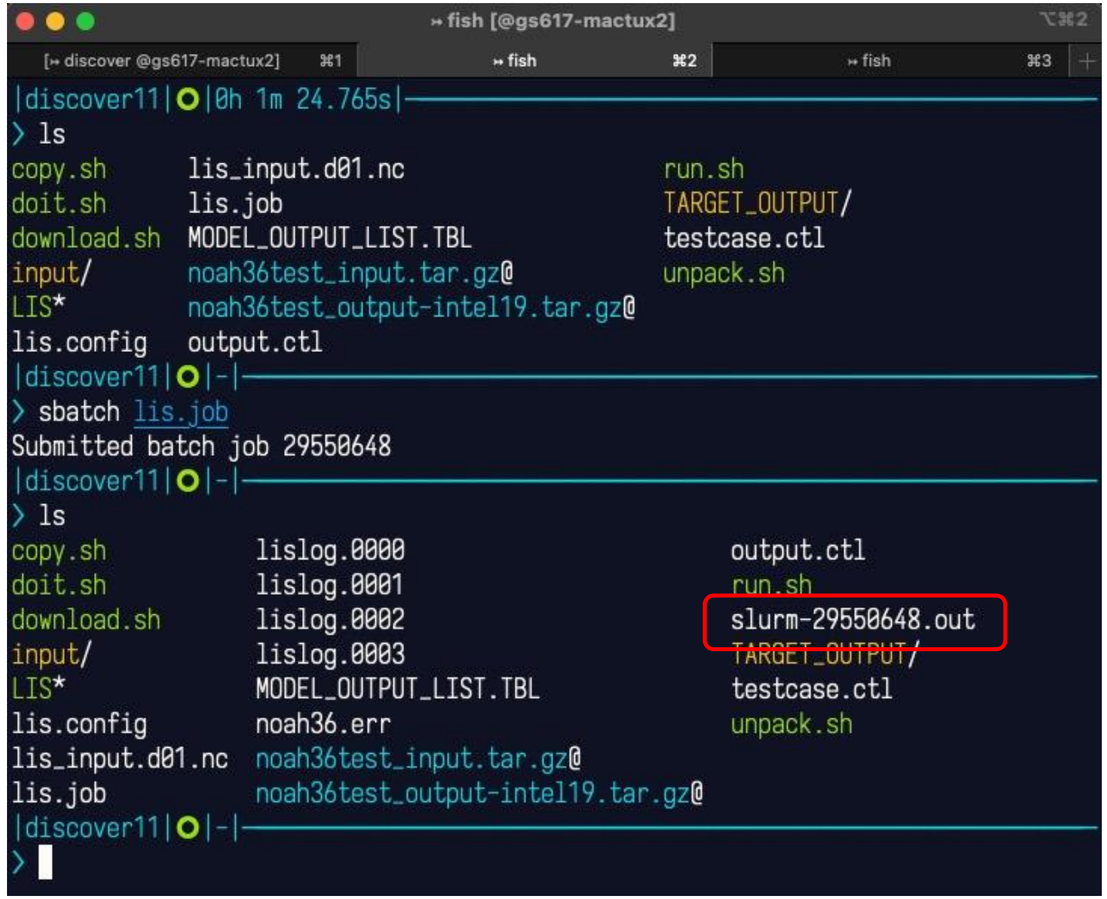
ulimit -s unlimited
mpirun -np 4 ./LIS --file lis.config >noah36.err 2>&1
```



Then I waited for the job to end. Did it end successfully?

Error messages

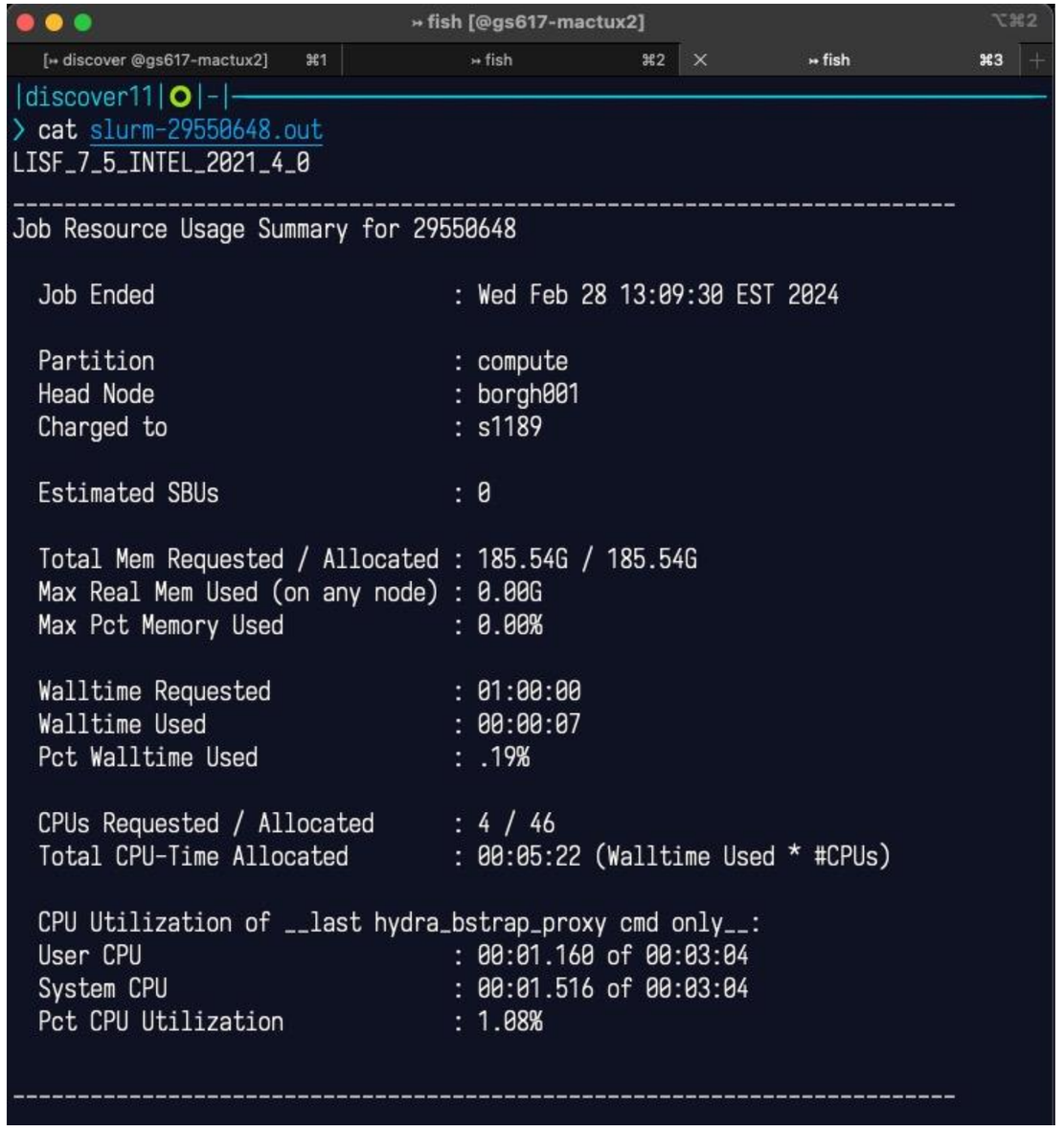
The *slurm-*.out* file contains messages from the queuing system.



Error messages

These messages are normal. This is not always the case.

So far, so good.

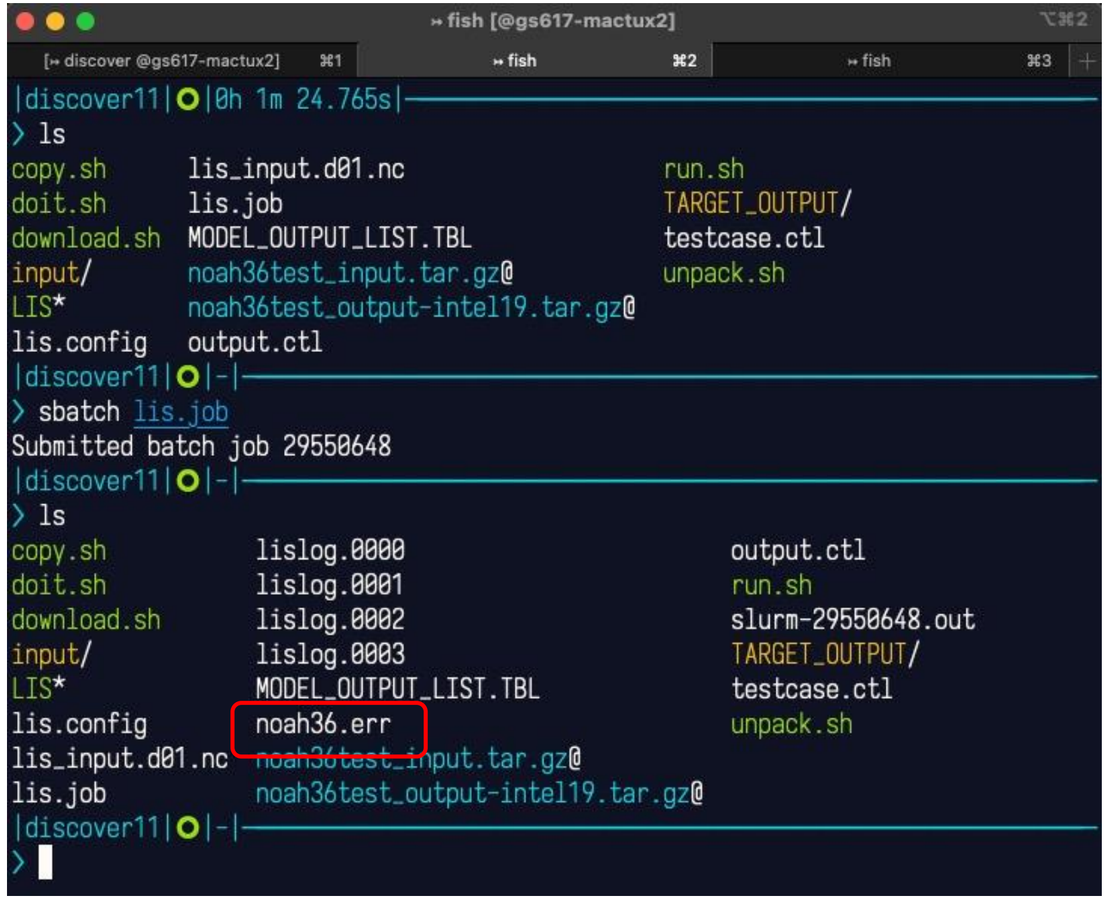


Error messages

My job script captured messages from the OS into this file named *noah36.err*.

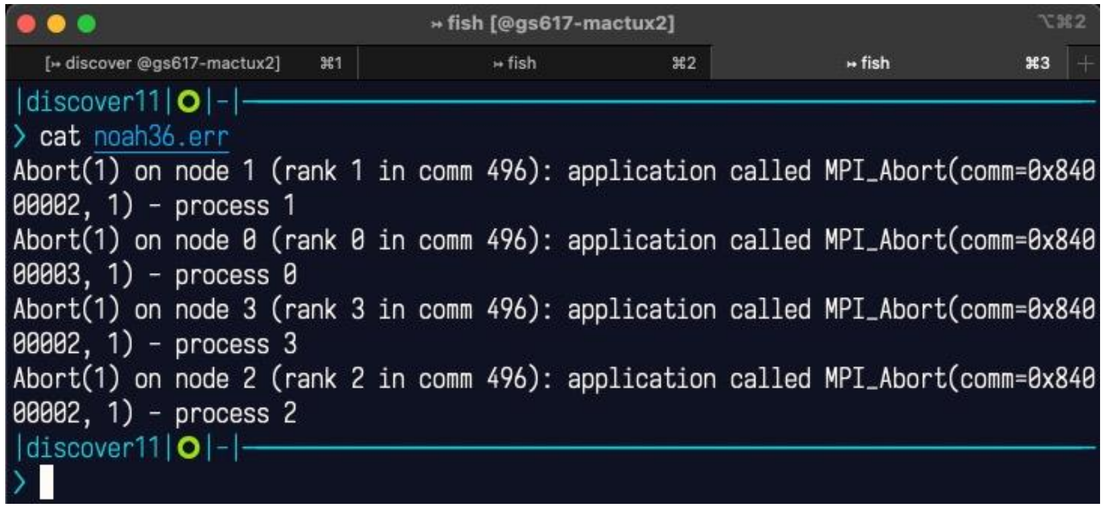
From my *lis.job* script:

```
mpirun -np 4 ./LIS --file lis.config >noah36.err 2>&1
```



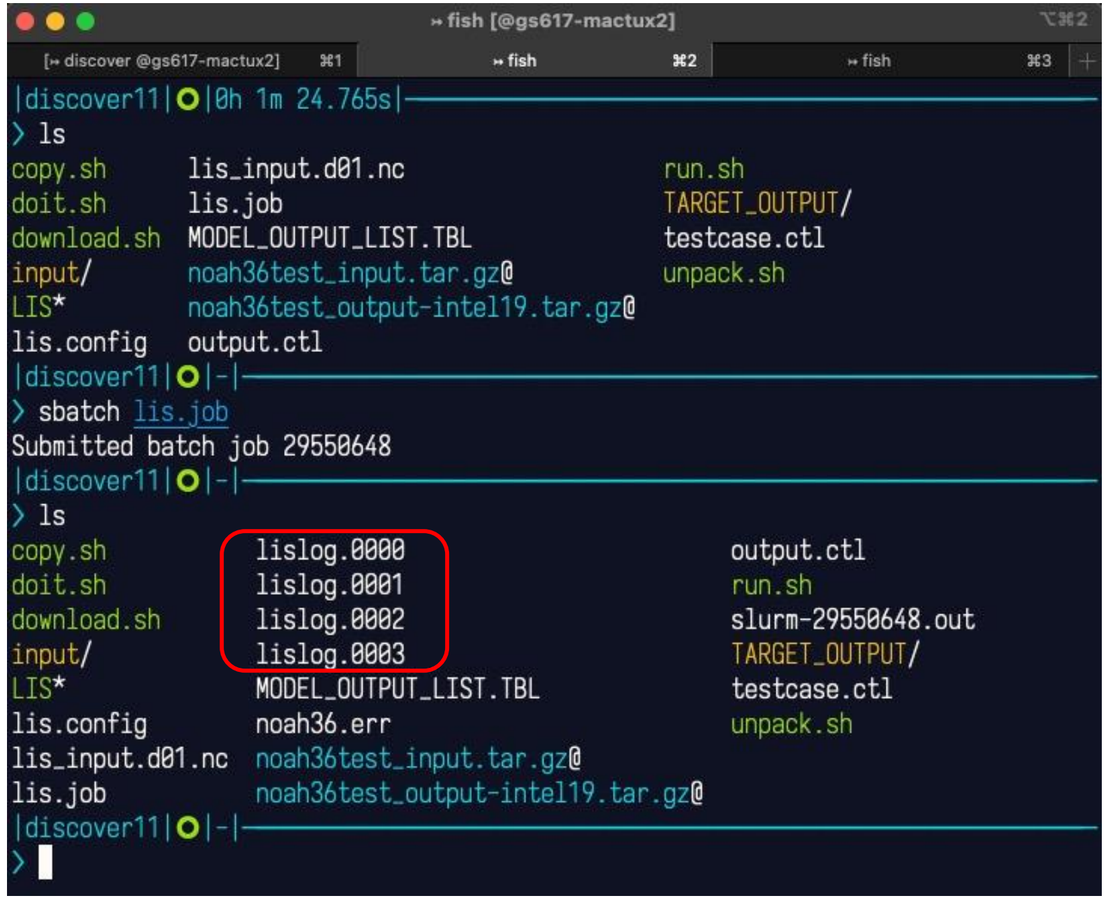
Error messages

Here we can see that my job aborted. But we do not know why.



Error messages

LIS, itself, writes log files, so you can see the processing that it did.



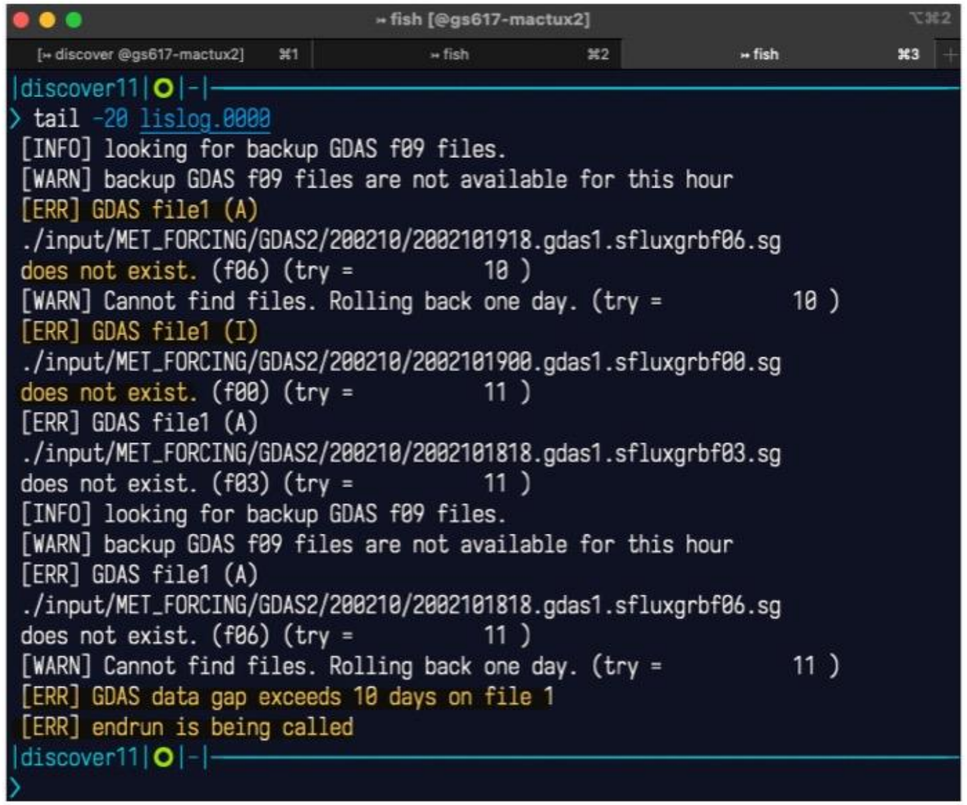
Error messages

Here I highlighted several error messages indicating that files cannot be found.

This is the problem. LIS could not find input data and aborted.

According to this log, one file that LIS could not find was:

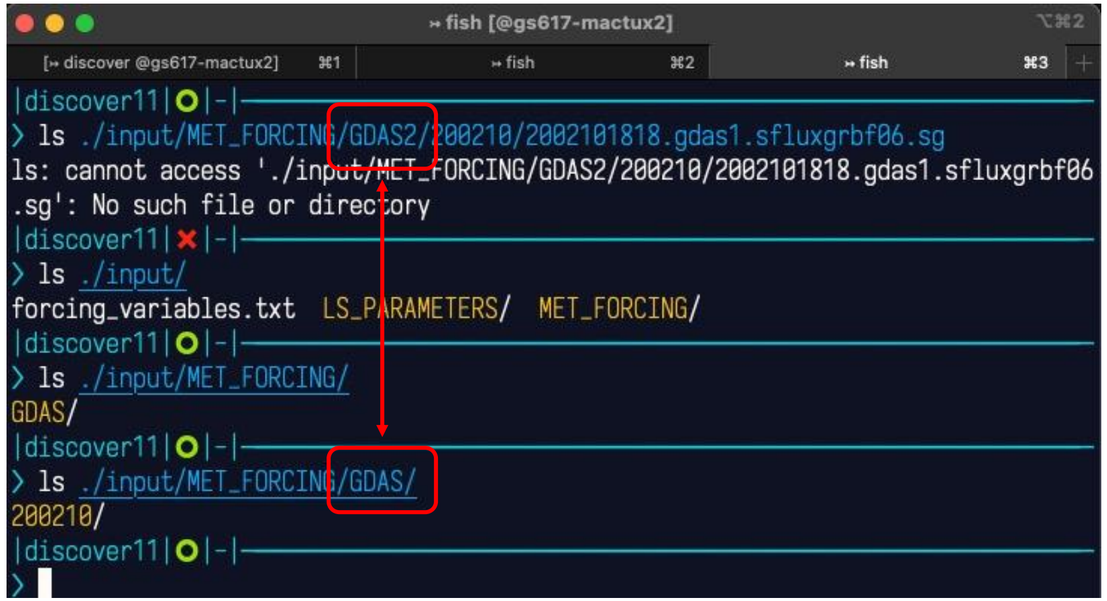
./input/MET_FORCING/GDAS2/200210/2002101818.gdas1.sfluxgrbf06.sg



Error messages

Looking for this missing file reveals the source of the problem.

LIS is looking for *./input/MET_FORCING/GDAS**2***, but the directory on disk is *./input/MET_FORCING/GDAS*. That **2** is a typo.

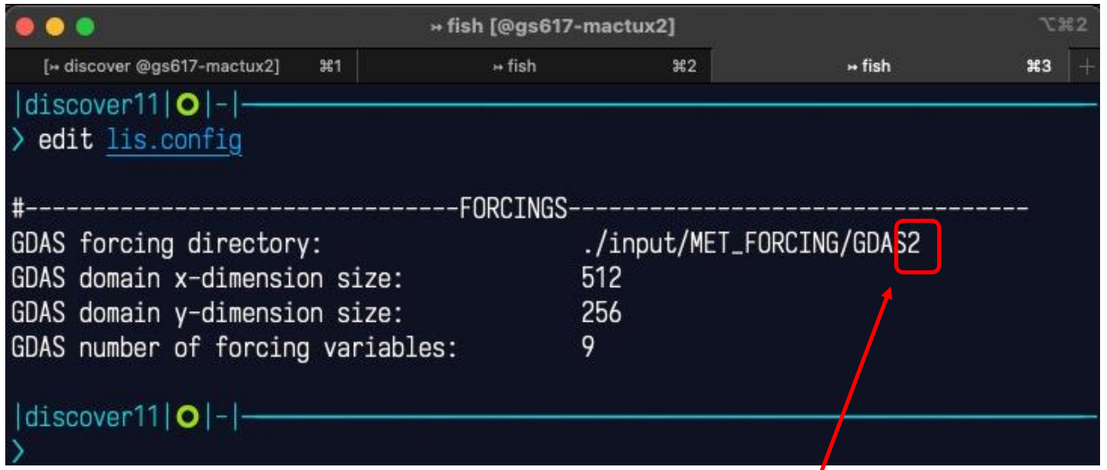


Error messages

To fix this problem, edit the *lis.config* file and correctly spell the name to the `GDAS forcing directory:` entry.

This should be

`GDAS forcing directory: ./input/MET_FORCING/GDAS`



Typo. Delete the 2.

Error messages

Sometimes, something goes wrong when running LIS. Between the messages reported by the operating system, the queuing system, the run-time in the executable, and LIS itself, you can track down the source of the problem and fix it.

Error Messages

Thank you.